

UNIVERSIDAD PERUANA LOS ANDES

Facultad de Ingeniería

Escuela Profesional de Ingeniería de Sistemas y Computación



Base de Datos II
ADMINISTRACIÓN ESENCIAL

Estudiante: Ramón Puente Eduardo Fredy
Docente: Ing. Raúl Fernández Bejarano

SEMANA 10

Huancayo, Junín, Perú — 2026

PROYECTO 1: CREACIÓN Y DISTRIBUCIÓN DE ARCHIVOS FÍSICOS DE LA BASE GRADOSTITULOS

1) Enunciado:

Crea la base de datos GradosTitulos con un archivo primario, un archivo secundario de datos para optimizar las consultas de documentos históricos, y el log de transacciones en rutas físicas dedicadas dentro del servidor. Posteriormente, realiza la consulta de los archivos físicos asociados para validar la distribución.

2) Script de la solución en T-SQL:

```
USE master;
GO

-- Validar existencia previa y desconectar usuarios
IF EXISTS (SELECT name FROM sys.databases WHERE name = N'GradosTitulos')
BEGIN
    ALTER DATABASE GradosTitulos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE GradosTitulos;
END
GO

-- Creación física distribuida (incluye grupo secundario para Documentos)
CREATE DATABASE GradosTitulos
ON PRIMARY (
    NAME = GradosTitulos_Data,
    FILENAME = 'D:\BaseDatos2026\GradosTitulos_data.mdf',
    SIZE = 20MB,
    MAXSIZE = 20GB,
    FILEGROWTH = 5MB
),
FILEGROUP DocumentosGroup (
    NAME = GradosTitulos_DocData,
    FILENAME = 'D:\BaseDatos2026\GradosTitulos_docdata.ndf',
    SIZE = 10MB,
    MAXSIZE = 10GB,
    FILEGROWTH = 2MB
)
LOG ON (
    NAME = GradosTitulos_Log,
    FILENAME = 'D:\BaseDatos2026\GradosTitulos_log.ldf',
    SIZE = 6MB,
    MAXSIZE = 2GB,
    FILEGROWTH = 2MB
);
GO

-- Consulta de verificación de archivos físicos
USE GradosTitulos;
GO
SELECT
    df.name AS NombreLogico,
    df.physical_name AS RutaFisica,
    df.type_desc AS TipoArchivo,
    fg.name AS GrupoArchivos,
    (df.size * 8) / 1024 AS TamanoActual_MB
FROM sys.database_files df
LEFT JOIN sys.filegroups fg ON df.data_space_id = fg.data_space_id;
GO
```

3) Justificación técnica de la solución aplicada:

La separación de archivos en un grupo primario (.mdf) y secundario (.ndf) permite aislar tablas operativas de alta transaccionalidad de aquellas que almacenan datos históricos pesados (como la tabla Documento.Diploma o Tramite.Fotografia). Esto disminuye la contención de I/O en los discos del servidor, optimizando los tiempos de respuesta de las cabezas de lectura/escritura y flexibilizando las estrategias de respaldo parcial.

4) Explicación de las buenas prácticas utilizadas en el proyecto:

Se implementa la cláusula SET SINGLE_USER WITH ROLLBACK IMMEDIATE antes del descarte de la base de datos para asegurar que ninguna conexión persistente bloquee la recreación del entorno. Además, se definieron parámetros

controlados de crecimiento (FILEGROWTH), evitando el uso de valores por defecto o porcentuales que degradan el rendimiento por fragmentación física.

Ejercicios Propuestos — Proyecto 1

Ejercicio 1: Incorporación de un grupo de archivos exclusivo para índices

Modifica la estructura de la base de datos GradosTitulos agregando un nuevo grupo de archivos llamado IdxGroup con un archivo físico secundario (.ndf). Esto servirá para almacenar de forma aislada todos los índices no agrupados de la base de datos.

```
USE master;
GO
ALTER DATABASE GradosTitulos
ADD FILEGROUP IdxGroup;
GO

ALTER DATABASE GradosTitulos
ADD FILE (
    NAME = GradosTitulos_IdxData,
    FILENAME = 'D:\BaseDatos2026\GradosTitulos_idx.ndf',
    SIZE = 5MB,
    MAXSIZE = 5GB,
    FILEGROWTH = 1MB
) TO FILEGROUP IdxGroup;
GO

-- Verificar filegroups
SELECT name, type_desc, is_default FROM sys.filegroups;
GO
```

Ejercicio 2: Reubicación de un Filegroup y consulta de espacio disponible

```
USE GradosTitulos;
GO
SELECT
    fg.name AS GrupoArchivos,
    df.name AS ArchivoLogico,
    (df.size * 8) / 1024 AS EspacioTotal_MB,
    ((df.size - FILEPROPERTY(df.name, 'SpaceUsed')) * 8) / 1024 AS EspacioLibre_MB,
    (FILEPROPERTY(df.name, 'SpaceUsed') * 8) / 1024 AS EspacioUsado_MB
FROM sys.database_files df
LEFT JOIN sys.filegroups fg ON df.data_space_id = fg.data_space_id
WHERE df.type_desc = 'ROWS_DATA';
GO
```

PROYECTO 2: AJUSTE DE CONFIGURACIÓN Y VALIDACIÓN DE PROPIEDADES DE GRADOSTITULOS

1) Enunciado:

Consulta las propiedades de colación actuales de la base de datos. Modifica la colación de GradosTitulos para asegurar el soporte nativo de caracteres en español (tildes y la letra Ñ). Adicionalmente, ajusta el crecimiento automático del archivo primario de datos de manera fija a 20 MB.

2) Script de la solución en T-SQL:

```
USE master;
GO

-- Consulta colación actual
SELECT name, collation_name, compatibility_level
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Poner en modo single user para modificar colación
ALTER DATABASE GradosTitulos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
GO

-- Modificar colación a Spanish_CI_AS (soporte completo de español)
```

```

ALTER DATABASE GradosTitulos COLLATE Modern_Spanish_CI_AS;
GO

-- Restaurar modo multi usuario
ALTER DATABASE GradosTitulos SET MULTI_USER;
GO

-- Ajustar crecimiento automático del archivo primario a 20MB fijo
ALTER DATABASE GradosTitulos
MODIFY FILE (
    NAME = GradosTitulos_Data,
    FILEGROWTH = 20MB
);
GO

-- Verificar cambios
SELECT name, collation_name FROM sys.databases WHERE name = 'GradosTitulos';
SELECT name, growth, is_percent_growth FROM sys.master_files WHERE database_id =
DB_ID('GradosTitulos');
GO

```

3) Justificación técnica:

La colación Modern_Spanish_CI_AS garantiza la ordenación correcta de caracteres especiales del español (ñ, á, é, í, ó, ú). CI indica Case-Insensitive y AS indica Accent-Sensitive, lo cual es ideal para un sistema universitario donde los apellidos y nombres deben ordenarse correctamente. Un FILEGROWTH fijo de 20 MB evita expansiones porcentuales que podrían fragmentar el disco con crecimientos desproporcionados.

4) Buenas prácticas:

Siempre se coloca la base en SINGLE_USER antes de cambiar la colación para evitar conflictos con sesiones activas. Se verifica el cambio con consultas al catálogo sys.databases y sys.master_files para confirmar que la configuración fue aplicada correctamente.

Ejercicios Propuestos — Proyecto 2

Ejercicio 1: Elevación del Nivel de Compatibilidad

```

USE master;
GO

-- Verificar nivel actual
SELECT name, compatibility_level FROM sys.databases WHERE name = 'GradosTitulos';
GO

-- Elevar a SQL Server 2022 (nivel 160)
ALTER DATABASE GradosTitulos SET COMPATIBILITY_LEVEL = 160;
GO

-- Verificar cambio
SELECT name, compatibility_level FROM sys.databases WHERE name = 'GradosTitulos';
GO

```

Ejercicio 2: Activación de PAGE_VERIFY CHECKSUM

```

ALTER DATABASE GradosTitulos SET PAGE_VERIFY CHECKSUM;
GO

-- Verificar
SELECT name, page_verify_option_desc
FROM sys.databases WHERE name = 'GradosTitulos';
GO

```

PROYECTO 3: DEFINICIÓN DE MODELO DE RECUPERACIÓN Y RESPALDO PARA GRADOSTITULOS

1) Enunciado:

Cambia de forma secuencial el modelo de recuperación de la base de datos GradosTitulos a SIMPLE y posteriormente a BULK_LOGGED. Explica el impacto de ambos entornos. Finalmente, cambia el modelo a FULL y ejecuta un respaldo completo (BACKUP DATABASE) hacia un directorio seguro de almacenamiento.

2) Script de la solución en T-SQL:

```

USE master;
GO

-- Cambio a modelo SIMPLE
ALTER DATABASE GradosTitulos SET RECOVERY SIMPLE;
GO
SELECT name, recovery_model_desc FROM sys.databases WHERE name = 'GradosTitulos';
GO

-- Cambio a modelo BULK_LOGGED
ALTER DATABASE GradosTitulos SET RECOVERY BULK_LOGGED;
GO
SELECT name, recovery_model_desc FROM sys.databases WHERE name = 'GradosTitulos';
GO

-- Cambio final a modelo FULL (requerido para respaldos de log)
ALTER DATABASE GradosTitulos SET RECOVERY FULL;
GO

-- Respaldo completo
BACKUP DATABASE GradosTitulos
TO DISK = 'D:\Backups\GradosTitulos_Full.bak'
WITH FORMAT, INIT,
    NAME = 'GradosTitulos - Respaldo Completo',
    DESCRIPTION = 'Backup completo Semana 10 BD2',
    STATS = 10;
GO

-- Verificar respaldo generado
RESTORE HEADERONLY FROM DISK = 'D:\Backups\GradosTitulos_Full.bak';
GO

```

3) Justificación técnica:

SIMPLE: El log de transacciones se trunca automáticamente en cada checkpoint. Ideal para entornos de desarrollo o bases sin requisitos de recuperación punto en el tiempo. No permite respaldos de log. **BULK_LOGGED:** Minimiza el registro de operaciones masivas (INSERT SELECT, BULK INSERT, bcp). Reduce el tamaño del log durante cargas de datos masivas, pero limita la recuperación a puntos exactos. **FULL:** Registra todas las transacciones de forma completa. Permite recuperación punto en el tiempo (Point-in-Time Recovery). Obligatorio en entornos de producción institucionales.

4) Buenas prácticas:

Se emplea WITH FORMAT, INIT para inicializar el archivo .bak evitando mezcla de backups en un mismo archivo. STATS=10 proporciona retroalimentación visual del progreso. Se recomienda verificar siempre el backup con RESTORE HEADERONLY o RESTORE VERIFYONLY.

Ejercicios Propuestos — Proyecto 3

Ejercicio 1: Respaldo diferencial de seguridad

```

BACKUP DATABASE GradosTitulos
TO DISK = 'D:\Backups\GradosTitulos_Diff.bak'
WITH DIFFERENTIAL, FORMAT, INIT,
    NAME = 'GradosTitulos - Respaldo Diferencial',
    STATS = 10;
GO

```

Ejercicio 2: Respaldo periódico del Log de Transacciones

```

-- Requiere modelo FULL o BULK_LOGGED
ALTER DATABASE GradosTitulos SET RECOVERY FULL;
GO

BACKUP LOG GradosTitulos
TO DISK = 'D:\Backups\GradosTitulos_Log.bak'
WITH FORMAT, INIT,
    NAME = 'GradosTitulos - Log de Transacciones',
    STATS = 10;
GO

```

PROYECTO 4: IMPLEMENTACIÓN DE ROLES Y USUARIOS PARA SEGURIDAD EN GRADOSTITULOS

1) Enunciado:

Crea un esquema de seguridad basado en roles integrados del sistema. Configura el login de servidor y usuario de base de datos VendedorQhatu (renombrado bajo el ecosistema institucional como OperadorTramite), asignándolo al rol db_datawriter para la gestión diaria del flujo administrativo. Asimismo, configura el usuario ConsultaCliente (renombrado como AuditorSunedu) vinculándolo exclusivamente al rol db_datareader.

2) Script de la solución en T-SQL:

```
USE master;
GO

-- Crear logins de servidor
IF NOT EXISTS (SELECT name FROM sys.server_principals WHERE name = 'OperadorTramite')
    CREATE LOGIN OperadorTramite WITH PASSWORD = 'OT@Segura2026!';
GO

IF NOT EXISTS (SELECT name FROM sys.server_principals WHERE name = 'AuditorSunedu')
    CREATE LOGIN AuditorSunedu WITH PASSWORD = 'AS@Segura2026!';
GO

USE GradosTitulos;
GO

-- Crear usuarios en la base de datos
CREATE USER OperadorTramite FOR LOGIN OperadorTramite;
GO

CREATE USER AuditorSunedu FOR LOGIN AuditorSunedu;
GO

-- Asignar roles
ALTER ROLE db_datawriter ADD MEMBER OperadorTramite;
GO

ALTER ROLE db_datareader ADD MEMBER AuditorSunedu;
GO

-- Verificar membresías
SELECT dp.name AS Usuario, r.name AS Rol
FROM sys.database_role_members drm
JOIN sys.database_principals dp ON drm.member_principal_id = dp.principal_id
JOIN sys.database_principals r ON drm.role_principal_id = r.principal_id
WHERE dp.name IN ('OperadorTramite', 'AuditorSunedu');
GO
```

3) Justificación técnica:

Los roles db_datawriter y db_datareader son roles predefinidos del motor SQL Server que simplifican la administración de permisos. db_datawriter otorga INSERT, UPDATE y DELETE sobre todas las tablas del esquema, mientras db_datareader concede SELECT sobre todas las tablas. El uso de roles en lugar de permisos individuales facilita la gestión centralizada y reduce el riesgo de configuraciones inconsistentes.

4) Buenas prácticas:

Las contraseñas cumplen la política de complejidad del servidor (mayúsculas, números, caracteres especiales). Se verifica la existencia del login antes de crear para evitar errores en entornos donde el script se ejecuta múltiples veces. Se utiliza sys.database_role_members para auditar los roles asignados.

Ejercicios Propuestos — Proyecto 4

Ejercicio 1: Creación de un Rol de Base de Datos personalizado (Custom Database Role)

```
USE GradosTitulos;
GO

-- Crear rol personalizado
CREATE ROLE RolSecretariaAcademica;
GO
```

```
-- Permisos SELECT en esquemas Academico y Tramite
GRANT SELECT ON SCHEMA::Academico TO RolSecretariaAcademica;
GRANT SELECT ON SCHEMA::Tramite TO RolSecretariaAcademica;

-- Permiso INSERT exclusivo en Tramite.Tramite
GRANT INSERT ON Tramite.Tramite TO RolSecretariaAcademica;
GO

-- Asignar el rol a un usuario de prueba
-- CREATE USER SecretariaUser FOR LOGIN SecretariaLogin;
-- ALTER ROLE RolSecretariaAcademica ADD MEMBER SecretariaUser;
GO
```

Ejercicio 2: Denegación explícita de acceso mediante DENY

```
USE GradosTitulos;
GO

-- Denegar acceso a tabla financiera sensible
DENY SELECT ON Tramite.Pago TO OperadorTramite;
GO

-- Verificar: DENY prevalece sobre cualquier rol
SELECT class_desc, permission_name, state_desc, grantee_principal_id
FROM sys.database_permissions
WHERE grantee_principal_id = USER_ID('OperadorTramite');
GO
```

PROYECTO 5: CONFIGURACIÓN GRANULAR DE PERMISOS EN EL MÓDULO ACADÉMICO DE GRADOSTITULOS

1) Enunciado:

Aplica el principio de mínimo privilegio de forma granular. Concede al usuario GerenteQhatu (actualizado a CoordinadorFacultad) acceso exclusivo de solo lectura (SELECT) a la vista analítica Evaluacion.V_NotaFinalSustentacion. En paralelo, revoca explícitamente al rol o usuario CajeroQhatu (actualizado a AsistenteTramite) el permiso de modificación (UPDATE) sobre la tabla crítica Tramite.Pago.

2) Script de la solución en T-SQL:

```
USE GradosTitulos;
GO

-- Crear usuarios si no existen
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'CoordinadorFacultad')
BEGIN
    CREATE LOGIN CoordinadorFacultad WITH PASSWORD = 'CF@2026Seguro!';
    CREATE USER CoordinadorFacultad FOR LOGIN CoordinadorFacultad;
END
GO

IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'AsistenteTramite')
BEGIN
    CREATE LOGIN AsistenteTramite WITH PASSWORD = 'AT@2026Seguro!';
    CREATE USER AsistenteTramite FOR LOGIN AsistenteTramite;
END
GO

-- Permiso granular: SELECT sobre la vista analítica
GRANT SELECT ON Evaluacion.V_NotaFinalSustentacion TO CoordinadorFacultad;
GO

-- Revocar UPDATE sobre tabla financiera critica
REVOKE UPDATE ON Tramite.Pago FROM AsistenteTramite;
GO

-- Verificar permisos aplicados
SELECT dp.name AS Usuario, p.permission_name, p.state_desc, OBJECT_NAME(p.major_id) AS Objeto
```

```
FROM sys.database_permissions p
JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
WHERE dp.name IN ('CoordinadorFacultad', 'AsistenteTramite');
GO
```

3) Justificación técnica:

El principio de mínimo privilegio dicta que cada usuario o rol debe tener únicamente los permisos estrictamente necesarios para su función. Conceder SELECT sobre una vista en lugar de la tabla base proporciona una capa adicional de seguridad: el usuario no puede conocer la estructura interna de las tablas ni acceder a columnas sensibles que no estén expuestas por la vista.

4) Buenas prácticas:

Se prefiere REVOKE sobre DENY cuando el objetivo es simplemente retirar un permiso previamente concedido, ya que DENY tiene mayor precedencia y podría bloquear accesos concedidos a través de roles. Se documenta cada cambio de permisos en un log de auditoría o comentario en el script.

Ejercicios Propuestos — Proyecto 5

Ejercicio 1: Permisos granulares a nivel de columnas (Column-Level Permissions)

```
USE GradosTitulos;
GO

-- Permiso solo sobre columnas no sensibles
GRANT SELECT ON Academico.Persona
    (IdPersona, Nombres, ApellidoPaterno, Correo)
TO PracticanteSistemas;
GO
```

Ejercicio 2: Concesión del permiso EXECUTE sobre Procedimientos Almacenados

```
USE GradosTitulos;
GO

GRANT EXECUTE ON Tramite.SP_RegistrarTramite TO OperadorTramite;
GO

-- Verificar
SELECT dp.name, p.permission_name, OBJECT_NAME(p.major_id) AS Objeto
FROM sys.database_permissions p
JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
WHERE dp.name = 'OperadorTramite';
GO
```

PROYECTO 6: IDENTIFICACIÓN Y SOLUCIÓN DE PROCESOS LENTOS Y BLOQUEOS EN GRADOSTITULOS

1) Enunciado:

Durante periodos masivos de carga académica del flujo de egresados, el servidor experimenta lentitud. Describe los indicadores clave de rendimiento a monitorizar mediante la interfaz gráfica del Activity Monitor. Simultáneamente, desarrolla una consulta avanzada en T-SQL utilizando vistas de administración dinámica (DMVs) para rastrear bloqueos de recursos en tiempo real en la base de datos.

2) Script de la solución en T-SQL:

```
USE GradosTitulos;
GO

-- =====
-- Consulta 1: Sesiones activas con bloqueos en tiempo real
-- =====

SELECT
    r.session_id AS SesionID,
    r.blocking_session_id AS BloqueadoPor,
    r.wait_type AS TipoEspera,
    r.wait_time / 1000 AS EsperaSegundos,
    r.status AS EstadoSesion,
    DB_NAME(r.database_id) AS BaseDatos,
    s.login_name AS Usuario,
    t.text AS ConsultaEjecutada
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.blocking_session_id > 0
ORDER BY r.wait_time DESC;
GO

-- =====
-- Consulta 2: Procesos activos por base de datos GradosTitulos
-- =====

SELECT
    s.session_id,
    s.login_name,
    s.status,
    r.cpu_time AS CPU_ms,
    r.total_elapsed_time AS TiempoTotal_ms,
    r.reads AS Lecturas,
    r.writes AS Escrituras,
    t.text AS Consulta
FROM sys.dm_exec_sessions s
JOIN sys.dm_exec_requests r ON s.session_id = r.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.database_id = DB_ID('GradosTitulos')
ORDER BY r.total_elapsed_time DESC;
GO
```

3) Justificación técnica:

Las DMVs (Dynamic Management Views) como sys.dm_exec_requests, sys.dm_exec_sessions y sys.dm_exec_sql_text permiten obtener información en tiempo real sobre el estado del motor de base de datos sin necesidad de herramientas externas. blocking_session_id > 0 filtra exclusivamente los procesos que están siendo bloqueados, permitiendo identificar el origen del bloqueo y la cadena de sesiones afectadas.

4) Buenas prácticas:

Se recomienda ejecutar estas consultas periódicamente durante ventanas de alta carga y almacenar los resultados en tablas de auditoría. El Activity Monitor de SSMS proporciona una vista gráfica equivalente con los indicadores: Processor Time, Waiting Tasks, Database I/O y Recent Expensive Queries.

Ejercicios Propuestos — Proyecto 6

Ejercicio 1: Identificación de consultas que consumen mayor tiempo de CPU

```
USE master;
GO

-- Top 5 consultas por consumo de CPU en caché
```

```

SELECT TOP 5
    qs.total_worker_time / 1000 AS CPU_Total_ms,
    qs.execution_count AS Ejecuciones,
    qs.total_worker_time / qs.execution_count / 1000 AS CPU_Promedio_ms,
    DB_NAME(qt.dbid) AS BaseDatos,
    SUBSTRING(qt.text, 1, 200) AS Consulta
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
WHERE qt.dbid = DB_ID('GradosTitulos')
ORDER BY qs.total_worker_time DESC;
GO

```

Ejercicio 2: Rastreo del uso de los índices en las consultas (Index Usage Stats)

```

USE GradosTitulos;
GO

-- Índices no utilizados en búsquedas pero que consumen espacio
SELECT
    OBJECT_NAME(i.object_id) AS Tabla,
    i.name AS NombreIndice,
    i.type_desc AS TipoIndice,
    ius.user_seeks AS Busquedas,
    ius.user_scans AS Escaneos,
    ius.user_lookups AS Lookups,
    ius.user_updates AS Actualizaciones
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats ius
    ON i.object_id = ius.object_id AND i.index_id = ius.index_id
    AND ius.database_id = DB_ID('GradosTitulos')
WHERE i.type_desc <> 'HEAP'
    AND ISNULL(ius.user_seeks, 0) = 0
    AND ISNULL(ius.user_scans, 0) = 0
ORDER BY ius.user_updates DESC;
GO

```

PROYECTO 7: AUTOMATIZACIÓN DE RESPALDOS Y LIMPIEZA PERIÓDICA DEL SISTEMA DE GRADOS

1) Enunciado:

Diseña los scripts transaccionales necesarios para automatizar tareas rutinarias a través de SQL Server Agent. El primer proceso ejecutará un respaldo completo programado diariamente de la base de datos. El segundo proceso se encargará de purgar semanalmente todos los registros de auditoría almacenados en la tabla de observaciones que superen los 15 días de antigüedad y cuyo estado ya se encuentre resuelto.

2) Script de la solución en T-SQL:

```

USE msdb;
GO

-- =====
-- JOB 1: Respaldo diario completo de GradosTitulos
-- =====
EXEC sp_add_job
    @job_name = N'BD_GradosTitulos_Backup_Diario';
GO

EXEC sp_add_jobstep
    @job_name = N'BD_GradosTitulos_Backup_Diario',
    @step_name = N'Ejecutar Backup Completo',
    @subsystem = N'TSQL',
    @command = N'
        BACKUP DATABASE GradosTitulos
        TO DISK = N''D:\Backups\GradosTitulos_' +
            CONVERT(VARCHAR, GETDATE(), 112) + ''.bak''
        WITH FORMAT, INIT, STATS = 10;';
    @database_name = N'master';
GO

```

```

-- Programar: todos los días a las 02:00 AM
EXEC sp_add_schedule
    @schedule_name = N'Diario_2AM',
    @freq_type = 4,          -- Diario
    @freq_interval = 1,
    @active_start_time = 020000;
GO

EXEC sp_attach_schedule
    @job_name = N'BD_GradosTitulos_Backup_Diario',
    @schedule_name = N'Diario_2AM';
GO

EXEC sp_add_jobserver
    @job_name = N'BD_GradosTitulos_Backup_Diario',
    @server_name = @@SERVERNAME;
GO

-- =====
-- JOB 2: Purga semanal de registros de auditoría resueltos
-- =====
EXEC sp_add_job
    @job_name = N'BD_GradosTitulos_Purga_Auditoria';
GO

EXEC sp_add_jobstep
    @job_name = N'BD_GradosTitulos_Purga_Auditoria',
    @step_name = N'Eliminar registros antiguos resueltos',
    @subsystem = N'TSQL',
    @command = N'
        USE GradosTitulos;
        DELETE FROM Auditoria.Observaciones
        WHERE FechaRegistro < DATEADD(DAY, -15, GETDATE())
        AND Estado = ''RESUELTO'';',
    @database_name = N'GradosTitulos';
GO

-- Programar: cada domingo a las 03:00 AM
EXEC sp_add_schedule
    @schedule_name = N'Semanal_Domingo_3AM',
    @freq_type = 8,          -- Semanal
    @freq_interval = 1,      -- Domingo
    @active_start_time = 030000;
GO

EXEC sp_attach_schedule
    @job_name = N'BD_GradosTitulos_Purga_Auditoria',
    @schedule_name = N'Semanal_Domingo_3AM';
GO

EXEC sp_add_jobserver
    @job_name = N'BD_GradosTitulos_Purga_Auditoria',
    @server_name = @@SERVERNAME;
GO

```

3) Justificación técnica:

SQL Server Agent permite la automatización de tareas administrativas recurrentes. El uso de CONVERT(VARCHAR, GETDATE(), 112) en el nombre del archivo .bak genera un nombre único con formato YYYYMMDD, evitando la sobrescritura de respaldos anteriores. La purga de registros de auditoría con más de 15 días resueltos mantiene el tamaño de la tabla controlado, mejorando el rendimiento de las consultas sobre ella.

4) Buenas prácticas:

Se programa el respaldo a las 02:00 AM en horario de baja carga del sistema. La purga de auditoría se ejecuta los domingos para no interferir con el flujo laboral. Se recomienda configurar alertas (sp_add_alert) para notificar al DBA ante fallos en estos jobs críticos.

Ejercicios Propuestos — Proyecto 7

Ejercicio 1: Notificación de alertas ante fallos en los Jobs del Agente

```
USE msdb;
GO

-- Crear operador DBA_Alertas
EXEC sp_add_operator
    @name = N'DBA_Alertas',
    @email_address = N'dba@upla.edu.pe';
GO

-- Alerta para Job fallido (evento 208: job failed)
EXEC sp_add_alert
    @name = N'Alerta_Backup_Fallido',
    @message_id = 0,
    @severity = 0,
    @job_name = N'BD_GradosTitulos_Backup_Diario',
    @notification_message = N'El Job de backup diario ha fallado. Verifique SSMS.';
GO

EXEC sp_add_notification
    @alert_name = N'Alerta_Backup_Fallido',
    @operator_name = N'DBA_Alertas',
    @notification_method = 1; -- Email
GO
```

Ejercicio 2: Automatización de la Reconstrucción de Índices Fragmentados

```
USE GradosTitulos;
GO

-- Script de mantenimiento: reconstruir índices con fragmentación > 30%
DECLARE @tabla NVARCHAR(256);
DECLARE @indice NVARCHAR(256);
DECLARE @frag FLOAT;

DECLARE curIndices CURSOR FOR
    SELECT
        QUOTENAME(OBJECT_SCHEMA_NAME(ips.object_id)) + '.' +
        QUOTENAME(OBJECT_NAME(ips.object_id)) AS Tabla,
        i.name AS Indice,
        ips.avg_fragmentation_in_percent AS Fragmentacion
    FROM sys.dm_db_index_physical_stats(DB_ID('GradosTitulos'), NULL, NULL, NULL,
    'SAMPLED') ips
    JOIN sys.indexes i ON ips.object_id = i.object_id AND ips.index_id = i.index_id
    WHERE ips.avg_fragmentation_in_percent > 30
    AND i.type_desc <> 'HEAP';

OPEN curIndices;
FETCH NEXT FROM curIndices INTO @tabla, @indice, @frag;

WHILE @@FETCH_STATUS = 0
BEGIN
    EXEC('ALTER INDEX [' + @indice + '] ON ' + @tabla + ' REBUILD;');
    FETCH NEXT FROM curIndices INTO @tabla, @indice, @frag;
END

CLOSE curIndices;
DEALLOCATE curIndices;
GO
```

PROYECTO 8: REESTRUCTURACIÓN FÍSICA Y ALTERACIÓN DE ESQUEMAS EN TABLAS DE PEDIDOS

1) Enunciado:

Con el fin de expandir el control analítico del proceso de sustentaciones y registros, adapta el ejercicio agregando de manera segura una columna denominada PrioridadAtencion de tipo entero (INT) a la estructura de la tabla Tramite.Tramite. Posteriormente, y emulando la corrección de requerimientos del sistema, elimina por completo la columna redundante Observaciones de dicha tabla.

2) Script de la solución en T-SQL:

```
USE GradosTitulos;
GO

-- Verificar estructura actual de la tabla
SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE, COLUMN_DEFAULT
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'Tramite' AND TABLE_NAME = 'Tramite'
ORDER BY ORDINAL_POSITION;
GO

-- Agregar columna PrioridadAtencion con valor por defecto
ALTER TABLE Tramite.Tramite
    ADD PrioridadAtencion INT NOT NULL
    CONSTRAINT DF_Tramite_Prioridad DEFAULT 1;
GO

-- Eliminar columna redundante (primero quitar constraint si existe)
IF EXISTS (
    SELECT 1 FROM sys.default_constraints
    WHERE name LIKE '%Observaciones%'
    AND parent_object_id = OBJECT_ID('Tramite.Tramite')
)
BEGIN
    DECLARE @constraintName NVARCHAR(256);
    SELECT @constraintName = name FROM sys.default_constraints
    WHERE parent_object_id = OBJECT_ID('Tramite.Tramite')
    AND COL_NAME(parent_object_id, parent_column_id) = 'Observaciones';
    EXEC('ALTER TABLE Tramite.Tramite DROP CONSTRAINT ' + @constraintName);
END
GO

ALTER TABLE Tramite.Tramite DROP COLUMN Observaciones;
GO

-- Verificar estructura actualizada
SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'Tramite' AND TABLE_NAME = 'Tramite'
ORDER BY ORDINAL_POSITION;
GO
```

3) Justificación técnica:

Al agregar una columna NOT NULL en una tabla existente con datos, es obligatorio definir un DEFAULT para no romper los registros actuales. Antes de eliminar una columna, se deben remover todas las restricciones (DEFAULT, CHECK, FK) que dependan de ella, de lo contrario el motor emite un error de dependencia. Se consulta sys.default_constraints para identificar dinámicamente el nombre del constraint.

4) Buenas prácticas:

Se consulta INFORMATION_SCHEMA.COLUMNS antes y después del ALTER TABLE para documentar el estado de la estructura. En entornos de producción, se recomienda ejecutar primero en un entorno de prueba y hacer un BACKUP antes de eliminar columnas con datos históricos.

Ejercicios Propuestos — Proyecto 8

Ejercicio 1: Incorporación de Restricciones CHECK complejas post-creación

```
USE GradosTitulos;
GO
```

```
-- Agregar CHECK: Monto siempre debe ser mayor a 0
ALTER TABLE Tramite.Pago
    ADD CONSTRAINT CHK_Pago_MontoPositivo
    CHECK (Monto > 0);
GO

-- Verificar constraint
SELECT name, definition FROM sys.check_constraints
WHERE parent_object_id = OBJECT_ID('Tramite.Pago');
GO
```

Ejercicio 2: Modificación del tipo de datos de una columna con validación de nulidad

```
USE GradosTitulos;
GO

-- Ampliar NroRecibo de VARCHAR(30) a VARCHAR(60) manteniendo NULL
ALTER TABLE Tramite.Pago
    ALTER COLUMN NroRecibo VARCHAR(60) NULL;
GO

-- Verificar el cambio
SELECT COLUMN_NAME, DATA_TYPE, CHARACTER_MAXIMUM_LENGTH, IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'Tramite'
    AND TABLE_NAME = 'Pago'
    AND COLUMN_NAME = 'NroRecibo';
GO
```

PROYECTO 9: IMPLEMENTACIÓN DE AUDITORÍA FORENSE AUTOMATIZADA MEDIANTE TRIGGERS

1) Enunciado:

Es necesario auditar por políticas de protección de datos personales toda eliminación física sobre el padrón de personas. Diseña e implementa una tabla de historial denominada `Academico.AuditoriaPersona`. Acto seguido, desarrolla un desencadenador (Trigger) de nivel de fila para el evento `DELETE` en la tabla `Academico.Persona`, el cual capturará los datos eliminados, el usuario del sistema que ejecutó la acción y el momento exacto del incidente.

2) Script de la solución en T-SQL:

```
USE GradosTitulos;
GO

-- =====
-- Tabla de auditoría forense
-- =====
IF NOT EXISTS (SELECT 1 FROM sys.objects WHERE name = 'AuditoriaPersona'
    AND SCHEMA_NAME(schema_id) = 'Academico')
BEGIN
    CREATE TABLE Academico.AuditoriaPersona (
        IdAuditoria INT IDENTITY(1,1) PRIMARY KEY,
        IdPersona INT,
        Nombres NVARCHAR(100),
        ApellidoPaterno NVARCHAR(100),
        ApellidoMaterno NVARCHAR(100),
        DNI VARCHAR(15),
        Correo VARCHAR(100),
        UsuarioSistema NVARCHAR(128),
        FechaEliminacion DATETIME DEFAULT GETDATE(),
        HostOrigen NVARCHAR(128)
    );
END
GO

-- =====
-- Trigger AFTER DELETE sobre Academico.Persona
-- =====
CREATE OR ALTER TRIGGER trg_Auditoria_DeletePersona
```

```

ON Academico.Persona
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO Academico.AuditoriaPersona (
        IdPersona, Nombres, ApellidoPaterno, ApellidoMaterno,
        DNI, Correo, UsuarioSistema, FechaEliminacion, HostOrigen
    )
    SELECT
        d.IdPersona,
        d.Nombres,
        d.ApellidoPaterno,
        d.ApellidoMaterno,
        d.DNI,
        d.Corrreo,
        SYSTEM_USER,
        GETDATE(),
        HOST_NAME()
    FROM deleted d;
END;
GO

-- Verificar creación del trigger
SELECT name, type_desc, is_disabled
FROM sys.triggers
WHERE parent_id = OBJECT_ID('Academico.Persona');
GO

```

3) Justificación técnica:

La tabla virtual deleted en SQL Server contiene todos los registros afectados por el DELETE antes de su eliminación física. El uso de SYSTEM_USER captura el login de SQL Server que ejecutó la transacción, y HOST_NAME() registra el nombre del equipo cliente. Este patrón cumple con los requisitos de trazabilidad exigidos por la Ley de Protección de Datos Personales (Ley N° 29733 en Perú).

4) Buenas prácticas:

Se utiliza CREATE OR ALTER TRIGGER para que el script sea idempotente (ejecutable múltiples veces sin errores). SET NOCOUNT ON evita el envío de mensajes de recuento de filas que podrían interferir con aplicaciones cliente. La tabla de auditoría debe estar en un filegroup separado o incluso en una base de datos dedicada en entornos de alta exigencia.

Ejercicios Propuestos — Proyecto 9

Ejercicio 1: Trigger de Auditoría para el control de Modificaciones (UPDATE)

```

USE GradosTitulos;
GO

-- Tabla de auditoría de pagos modificados
IF NOT EXISTS (SELECT 1 FROM sys.objects WHERE name = 'AuditoriaPago'
               AND SCHEMA_NAME(schema_id) = 'Tramite')
BEGIN
    CREATE TABLE Tramite.AuditoriaPago (
        IdAuditoria INT IDENTITY(1,1) PRIMARY KEY,
        IdPago INT,
        MontoAnterior DECIMAL(10,2),
        MontoNuevo DECIMAL(10,2),
        UsuarioModifico NVARCHAR(128),
        FechaModificacion DATETIME DEFAULT GETDATE()
    );
END
GO

-- Trigger AFTER UPDATE en Tramite.Pago
CREATE OR ALTER TRIGGER TR_Auditoria_ModificarPago
ON Tramite.Pago
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

```

```

INSERT INTO Tramite.AuditoriaPago (
    IdPago, MontoAnterior, MontoNuevo, UsuarioModifico, FechaModificacion
)
SELECT
    i.IdPago,
    d.Monto AS MontoAnterior,
    i.Monto AS MontoNuevo,
    SYSTEM_USER,
    GETDATE()
FROM inserted i
JOIN deleted d ON i.IdPago = d.IdPago
WHERE i.Monto <> d.Monto;
END;
GO

```

Ejercicio 2: Trigger de control de seguridad institucional (Restricción de Horarios)

```

USE GradosTitulos;
GO

-- Trigger INSTEAD OF que bloquea DELETE/UPDATE en fines de semana
CREATE OR ALTER TRIGGER TR_Seguridad_Horario_Docente
ON Academico.Docente
INSTEAD OF DELETE, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Verificar si es sábado (7) o domingo (1)
    IF DATEPART(WEEKDAY, GETDATE()) IN (1, 7)
    BEGIN
        RAISERROR(
            'Operación bloqueada: No se permiten modificaciones o eliminaciones',
            16, 1
        );
        ROLLBACK TRANSACTION;
        RETURN;
    END

    -- Si es día hábil, ejecutar la operación original
    -- Para UPDATE:
    IF EXISTS (SELECT 1 FROM inserted)
    BEGIN
        UPDATE d
        SET d.NombreDocente = i.NombreDocente -- Reemplazar con columnas reales
        FROM Academico.Docente d
        JOIN inserted i ON d.IdDocente = i.IdDocente;
    END
    ELSE
    BEGIN
        -- Para DELETE:
        DELETE FROM Academico.Docente
        WHERE IdDocente IN (SELECT IdDocente FROM deleted);
    END
END;
GO

```


PROYECTO 10: SIMULACIÓN DE RECUPERACIÓN Y RESTAURACIÓN COMPLETA ANTE CONTINGENCIAS DE DATOS

1) Enunciado:

Simula un escenario catastrófico provocado por error humano mediante el borrado accidental de datos críticos en el catálogo institucional. Posteriormente, ejecuta la restauración completa de la base de datos GradosTitulos utilizando el respaldo de seguridad (.bak) generado previamente en el Proyecto 3. Finaliza validando la correcta restitución y consistencia de los datos mediante una consulta de verificación.

2) Script de la solución en T-SQL:

```
-- =====
-- FASE 1: Simulación del desastre (borrado accidental)
-- =====

USE GradosTitulos;
GO

-- Registrar conteo antes del borrado
SELECT 'Antes del borrado' AS Momento, COUNT(*) AS TotalPersonas
FROM Academico.Persona;
GO

-- Simular borrado accidental de toda la tabla
DELETE FROM Academico.Persona;
GO

-- Verificar daño
SELECT 'Después del borrado' AS Momento, COUNT(*) AS TotalPersonas
FROM Academico.Persona;
GO

-- =====
-- FASE 2: Restauración desde el respaldo del Proyecto 3
-- =====

USE master;
GO

-- Desconectar usuarios activos
ALTER DATABASE GradosTitulos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
GO

-- Restauración completa
RESTORE DATABASE GradosTitulos
FROM DISK = 'D:\Backups\GradosTitulos_Full.bak'
WITH REPLACE,
    MOVE 'GradosTitulos_Data' TO 'D:\BaseDatos2026\GradosTitulos_data.mdf',
    MOVE 'GradosTitulos_DocData' TO 'D:\BaseDatos2026\GradosTitulos_docdata.ndf',
    MOVE 'GradosTitulos_Log' TO 'D:\BaseDatos2026\GradosTitulos_log.ldf',
    STATS = 10,
    RECOVERY;
GO

-- Restaurar modo multiusuario
ALTER DATABASE GradosTitulos SET MULTI_USER;
GO

-- =====
-- FASE 3: Validación de consistencia post-restauración
-- =====

USE GradosTitulos;
GO

-- Verificar datos restaurados
SELECT 'Post-restauración' AS Momento, COUNT(*) AS TotalPersonas
FROM Academico.Persona;
GO

-- Verificar integridad de la base de datos
```

```
DBCC CHECKDB('GradosTitulos') WITH NO_INFOMSGS;
GO

-- Consulta de verificación de archivos físicos restaurados
SELECT
    df.name AS NombreLogico,
    df.physical_name AS RutaFisica,
    df.state_desc AS EstadoArchivo,
    (df.size * 8) / 1024 AS TamanoMB
FROM sys.database_files df;
GO
```

3) Justificación técnica de la solución aplicada:

La cláusula WITH REPLACE en el RESTORE fuerza la sobreescritura de la base existente, incluso si hay diferencias en la estructura de archivos. La cláusula MOVE es fundamental cuando las rutas físicas difieren entre el servidor de origen del backup y el servidor de restauración. DBCC CHECKDB realiza una auditoría completa de integridad estructural y lógica de páginas de datos e índices, confirmando que no existen errores ocultos en el almacenamiento.

4) Explicación de las buenas prácticas utilizadas en el proyecto:

Se implementa la cláusula SET SINGLE_USER WITH ROLLBACK IMMEDIATE antes del descarte de la base de datos para asegurar que ninguna conexión persistente bloquee la recreación del entorno. Además, se definieron parámetros controlados de crecimiento (FILEGROWTH), evitando el uso de valores por defecto o porcentuales que degradan el rendimiento por fragmentación física. La ejecución de DBCC CHECKDB al finalizar es una práctica estándar de DBA para garantizar la integridad de la restauración ante cualquier corrupción silenciosa.

Ejercicios Propuestos — Proyecto 10

Ejercicio 1: Restauración parcial de tablas mediante exportación e inserción controlada

```
-- Usando Cross-Database Queries desde la copia de respaldo montada
-- como GradosTitulos_Auxiliar

USE GradosTitulos;
GO

-- Recuperar registros del catálogo desde la base auxiliar (montada como NORECOVERY)
INSERT INTO Catalogo.EstadoTramite (
    IdEstado, Descripcion, Activo
)
SELECT
    IdEstado, Descripcion, Activo
FROM GradosTitulos_Auxiliar.Catalogo.EstadoTramite
WHERE IdEstado NOT IN (SELECT IdEstado FROM Catalogo.EstadoTramite);
GO

-- Verificar registros recuperados
SELECT COUNT(*) AS RegistrosRecuperados FROM Catalogo.EstadoTramite;
GO
```

Ejercicio 2: Verificación de la consistencia física con DBCC CHECKDB

```
USE master;
GO

-- Auditoría completa de integridad de GradosTitulos
DBCC CHECKDB('GradosTitulos')
WITH
    ALL_ERRORMSG,      -- Mostrar todos los errores sin límite
    EXTENDED_LOGICAL_CHECKS, -- Verificaciones lógicas extendidas
    NO_INFOMSGS;      -- Suprimir mensajes informativos (solo errores)
GO

-- Verificar resultado: si no hay output de error, la DB es consistente
-- Para ver el resultado en tabla de historial:
SELECT TOP 10
    database_id,
    last_clean_run_date,
    is_online
FROM sys.databases
WHERE name = 'GradosTitulos';
GO
```